

LABSHEET - 8**HIMANI CHAUDHARY****BTECH(CSE) 3RD YEAR****CU240250967****SECTION - B****ROLL NO. - 24****COMPUTER VISION**

```
# =====
# EXPERIMENT NO. 8
# APPLICATION OF OPTICAL FLOW FOR REAL-TIME OBJECT TRACKING
# AND MOTION ANALYSIS
#
# Shi-Tomasi + Lucas-Kanade
# Farneback Dense Optical Flow
# GOOGLE COLAB - ONE CELL
# =====

import cv2
import numpy as np
import os
import math
from IPython.display import display, Video
from google.colab import files

# =====
# 1. VIDEO PATH
# =====

VIDEO_PATH = "/content/WhatsApp Video 2026-09-10 at 9.28.37 AM.mp4"
# =====
# 2. CHECK VIDEO
# =====

if not os.path.exists(VIDEO_PATH):
    raise FileNotFoundError(
        "Video not found!\nCheck this path:\n" + VIDEO_PATH
    )

print("Video found successfully!")
print("Path:", VIDEO_PATH)

# =====
# 3. OPEN VIDEO
# =====

cap = cv2.VideoCapture(VIDEO_PATH)

if not cap.isOpened():
    raise RuntimeError("OpenCV could not open the video.")

fps = cap.get(cv2.CAP_PROP_FPS)

if fps <= 0:
    fps = 25

total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

# =====
# 4. READ FIRST FRAME
# =====

ret, old_frame = cap.read()

if not ret or old_frame is None:
    cap.release()
    raise RuntimeError("Could not read first frame.")

# =====
# 5. RESIZE
# =====

MAX_WIDTH = 700

def resize_frame(frame):
```

```

h, w = frame.shape[:2]

if w > MAX_WIDTH:

    scale = MAX_WIDTH / w

    new_w = int(w * scale)
    new_h = int(h * scale)

    frame = cv2.resize(
        frame,
        (new_w, new_h)
    )

    return frame

old_frame = resize_frame(old_frame)

FRAME_HEIGHT = old_frame.shape[0]
FRAME_WIDTH = old_frame.shape[1]

old_gray = cv2.cvtColor(
    old_frame,
    cv2.COLOR_BGR2GRAY
)

# =====
# 6. SHI-TOMASI PARAMETERS
# =====

feature_params = {
    "maxCorners": 200,
    "qualityLevel": 0.3,
    "minDistance": 7,
    "blockSize": 7
}

# =====
# 7. INITIAL FEATURE POINTS
# =====

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    mask=None,
    **feature_params
)

# =====
# 8. LUCAS-KANADE PARAMETERS
# =====

lk_params = {
    "winSize": (15, 15),
    "maxLevel": 2,
    "criteria": (
        cv2.TERM_CRITERIA_EPS |
        cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
}

# =====
# 9. TRAJECTORY MASK
# =====

mask = np.zeros_like(old_frame)

# =====
# 10. VARIABLES
# =====

previous_center = None
total_displacement = 0.0
frame_count = 0

# =====
# 11. OUTPUT VIDEO
# =====

output_path = "/content/optical_flow_output.mp4"

```

```

fourcc = cv2.VideoWriter_fourcc(*"mp4v")

out = cv2.VideoWriter(
    output_path,
    fourcc,
    fps,
    (
        FRAME_WIDTH * 3,
        FRAME_HEIGHT
    )
)

if not out.isOpened():
    cap.release()
    raise RuntimeError("Could not create output video.")

# =====
# 12. PROCESS VIDEO
# =====

print("\n=====")
print("      PROCESSING VIDEO")
print("=====")

while True:

    ret, frame = cap.read()

    if not ret:
        break

    # -----
    # Resize
    # -----

    frame = resize_frame(frame)

    frame = cv2.resize(
        frame,
        (FRAME_WIDTH, FRAME_HEIGHT)
    )

    # -----
    # Grayscale
    # -----

    frame_gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )

    # =====
    # PART 1
    # SHI-TOMASI + LUCAS-KANADE
    # =====

    sparse_output = frame.copy()

    displacement = 0.0
    direction = 0.0

    if p0 is not None and len(p0) > 0:

        p1, status, error = cv2.calcOpticalFlowPyrLK(
            old_gray,
            frame_gray,
            p0,
            None,
            **lk_params
        )

        if p1 is not None and status is not None:

            p1_flat = p1.reshape(-1, 2)
            p0_flat = p0.reshape(-1, 2)

            status_flat = status.reshape(-1)

            good_new = p1_flat[status_flat == 1]
            good_old = p0_flat[status_flat == 1]

            if len(good_new) > 0:

```

```

if len(good_new) > 0:

    # -----
    # CENTER
    # -----

    center_x = int(
        np.mean(good_new[:, 0])
    )

    center_y = int(
        np.mean(good_new[:, 1])
    )

    current_center = (
        center_x,
        center_y
    )

    # -----
    # DISPLACEMENT
    # -----

    if previous_center is not None:

        dx = (
            current_center[0]
            - previous_center[0]
        )

        dy = (
            current_center[1]
            - previous_center[1]
        )

        displacement = math.sqrt(
            dx * dx + dy * dy
        )

        total_displacement += displacement

    # -----
    # DIRECTION
    # -----

    if dx != 0 or dy != 0:

        direction = math.degrees(
            math.atan2(
                -dy,
                dx
            )
        )

        if direction < 0:
            direction += 360

    # -----
    # DRAW TRACKED POINTS
    # -----

    for new, old in zip(
        good_new,
        good_old
    ):

        a = int(new[0])
        b = int(new[1])

        c = int(old[0])
        d = int(old[1])

        # Trajectory
        mask = cv2.line(
            mask,
            (c, d),
            (a, b),
            (0, 255, 0),
            2
        )

        # Feature point
        cv2.circle(

```

```

        sparse_output,
        (a, b),
        4,
        (0, 0, 255),
        -1
    )

    # Motion arrow
    cv2.arrowedLine(
        sparse_output,
        (c, d),
        (a, b),
        (255, 0, 0),
        1,
        tipLength=0.3
    )

# -----
# CENTER POINT
# -----

cv2.circle(
    sparse_output,
    current_center,
    8,
    (255, 0, 255),
    -1
)

# Add trajectory
sparse_output = cv2.add(
    sparse_output,
    mask
)

# -----
# TEXT
# -----

cv2.putText(
    sparse_output,
    "Shi-Tomasi + Lucas-Kanade",
    (15, 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.65,
    (0, 255, 255),
    2
)

cv2.putText(
    sparse_output,
    f"Tracked Points: {len(good_new)}",
    (15, 60),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.55,
    (0, 255, 255),
    2
)

cv2.putText(
    sparse_output,
    f"Displacement: {displacement:.2f} px",
    (15, 90),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.55,
    (0, 255, 255),
    2
)

cv2.putText(
    sparse_output,
    f"Direction: {direction:.1f} deg",
    (15, 120),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.55,
    (0, 255, 255),
    2
)

# Update center
previous_center = current_center

# Update points

```

```

        p0 = good_new.reshape(
            -1,
            1,
            2
        ).astype(np.float32)

    else:

        p0 = cv2.goodFeaturesToTrack(
            frame_gray,
            mask=None,
            **feature_params
        )

    else:

        p0 = cv2.goodFeaturesToTrack(
            frame_gray,
            mask=None,
            **feature_params
        )

    else:

        p0 = cv2.goodFeaturesToTrack(
            frame_gray,
            mask=None,
            **feature_params
        )

# =====
# PART 2
# FARNEBACK DENSE OPTICAL FLOW
# =====

flow = cv2.calcOpticalFlowFarneback(
    old_gray,
    frame_gray,
    None,
    0.5,
    3,
    15,
    3,
    5,
    1.2,
    0
)

# -----
# MAGNITUDE + ANGLE
# -----

magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

# -----
# HSV
# -----

hsv = np.zeros(
    (
        FRAME_HEIGHT,
        FRAME_WIDTH,
        3
    ),
    dtype=np.uint8
)

hsv[..., 1] = 255

hsv[..., 0] = (
    angle * 180 / np.pi / 2
).astype(np.uint8)

hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

```

```

# -----
# Convert to BGR
# -----

dense_output = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2BGR
)

# -----
# Dense Flow Text
# -----

cv2.putText(
    dense_output,
    "Farneback Dense Optical Flow",
    (15, 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.65,
    (255, 255, 255),
    2
)

# =====
# ORIGINAL VIDEO
# =====

original_output = frame.copy()

cv2.putText(
    original_output,
    "Original Video",
    (15, 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.75,
    (255, 255, 255),
    2
)

# =====
# COMBINE ALL THREE
# =====

combined = np.hstack(
    (
        original_output,
        sparse_output,
        dense_output
    )
)

# =====
# WRITE OUTPUT
# =====

out.write(combined)

# =====
# UPDATE FRAME
# =====

old_gray = frame_gray.copy()

frame_count += 1

# =====
# 13. RELEASE
# =====

cap.release()
out.release()

# =====
# 14. FINAL OUTPUT
# =====

print("\n=====")
print("      EXPERIMENT COMPLETED")
print("=====")
print("Frames Processed   :", frame_count)
print(

```

```

        "Total Displacement :",
        f"{total_displacement:.2f} pixels"
    )
print("Output File      :", output_path)
print("=====")

# =====
# 15. DISPLAY OUTPUT
# =====

print("\nFINAL OUTPUT VIDEO:")

display(
    Video(
        output_path,
        embed=True,
        width=1200
    )
)

# =====
# 16. DOWNLOAD OUTPUT
# =====

print("\nIf you want to download the output, run:")
# Convert output video to browser-compatible MP4

import subprocess
from IPython.display import Video, display

input_video = "/content/optical_flow_output.mp4"
fixed_video = "/content/optical_flow_output_fixed.mp4"

subprocess.run([
    "ffmpeg",
    "-y",
    "-i", input_video,
    "-vcodec", "libx264",
    "-pix_fmt", "yuv420p",
    "-movflags", "+faststart",
    fixed_video
], stdout=subprocess.PIPE, stderr=subprocess.PIPE)

print("Conversion completed!")
print("Fixed output:", fixed_video)

display(
    Video(
        fixed_video,
        embed=True,
        width=1200
    )
)
files.download(output_path)
print("Himani Chaudhary")
print("CU240250967")

```

Video found successfully!
Path: /content/WhatsApp Video 2026-09-10 at 9.28.37 AM.mp4

=====
PROCESSING VIDEO
=====

=====
EXPERIMENT COMPLETED
=====

Frames Processed : 22
Total Displacement : 10.66 pixels
Output File : /content/optical_flow_output.mp4
=====

FINAL OUTPUT VIDEO:

0:00



If you want to download the output, run:
Conversion completed!
Fixed output: /content/optical_flow_output_fixed.mp4

0:00 / 0:00